

EXERCISE 12.16

There's an interesting consequence of being able to turn any traversable functor into a *reversed* list—we can write, once and for all, a function to reverse any traversable functor! Write this function, and think about what it means for `List`, `Tree`, and other traversable functors.

```
def reverse[A](fa: F[A]): F[A]
```

It should obey the following law, for all `x` and `y` of the appropriate types:

```
toList(reverse(x)) ++ toList(reverse(y)) ==
reverse(toList(y) ++ toList(x))
```

EXERCISE 12.17

Use `mapAccum` to give a default implementation of `foldLeft` for the `Traverse` trait.

12.7.3 Combining traversable structures

It's the nature of a traversal that it must preserve the shape of its argument. This is both its strength and its weakness. This is well demonstrated when we try to combine two structures into one.

Given `Traverse[F]`, can we combine a value of some type `F[A]` and another of some type `F[B]` into an `F[C]`? We could try using `mapAccum` to write a generic version of `zip`.

Listing 12.12 Combining two different structure types

```
def zip[A,B](fa: F[A], fb: F[B]): F[(A, B)] =
  (mapAccum(fa, toList(fb)) {
    case (a, Nil) => sys.error("zip: Incompatible shapes.")
    case (a, b :: bs) => ((a, b), bs)
  })._1
```

Note that this version of `zip` is unable to handle arguments of different “shapes.” For example, if `F` is `List`, then it can't handle lists of different lengths. In this implementation, the list `fb` must be at least as long as `fa`. If `F` is `Tree`, then `fb` must have at least the same number of branches as `fa` at every level.

We can change the generic `zip` slightly and provide two versions so that the shape of one side or the other is dominant.